

Musterklausur Programmierung II

Daniel Appenmaier, Juli 2026

Aufgabe	Maximale Punktzahl	Erreichte Punktzahl
1	17	
2	16	
3	19	
Gesamt	52	

Hinweise

- Die in dieser Klausur verwendeten Personenbezeichnungen beziehen sich – sofern nicht anders kenntlich gemacht – auf alle Geschlechter
- Pakete und Klassenimporte müssen nicht angegeben werden
- Die Klassen und Annotationen der Bibliotheken **Lombok**, **JUnit 5** und **Mockito** dürfen verwendet werden
- Es kann davon ausgegangen werden, dass sämtliche Klassen entsprechende Implementierungen der Object-Methoden besitzen
- So nicht anders angegeben sollen Konstruktoren, Setter, Getter sowie die Object-Methoden wie in der Vorlesung gezeigt implementiert werden
- Die Konsolenausgaben-Methoden der Klasse **PrintStream** dürfen sinnvoll gekürzt geschrieben werden (zum Beispiel **sysout(„Hello World“)** statt **System.out.println(„Hello World“)**)
- Methoden- und Attributsbezeichner dürfen sinnvoll gekürzt geschrieben werden (zum Beispiel **getLWMCP()** statt **getLectureWithMostCreditPoints()**)

Wörterbuch

Englisch	Deutsch	Englisch	Deutsch
description	Beschreibung	robot	Roboter
fabrication date	Fabrikationsdatum	serial number	Seriennummer
fabrication year	Fabrikationsjahr	series	Serie
factory	Fabrik	terminator	Terminator
humanoid	menschenähnlich	weight	Gewicht

Aufgabe 1 (17 Punkte)

Erstelle die Klassen **Terminator** (7 Punkte) und **RobotFactory** (10 Punkte) anhand des abgebildeten Klassendiagramms.

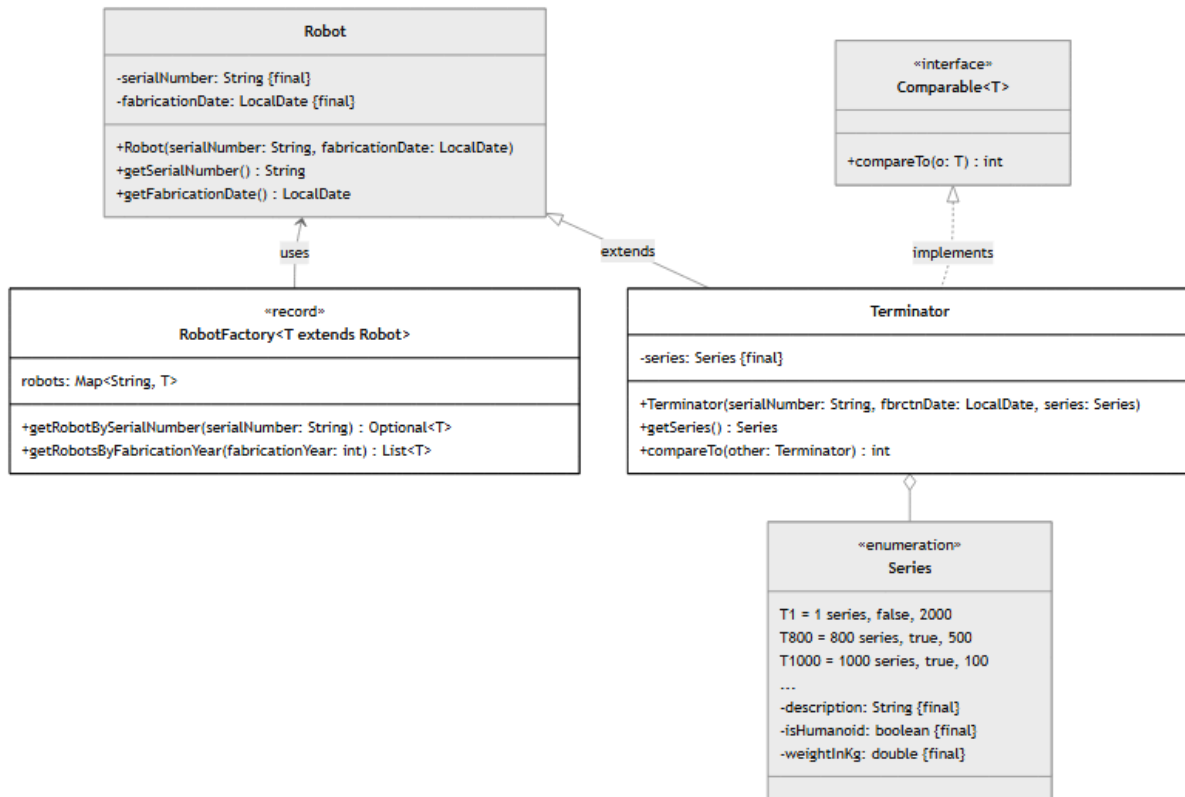


Abbildung 1: Klassendiagramm Aufgabe 1

Hinweise zur Klasse *Terminator*

Die Methode `int compareTo(other: Terminator)` soll so implementiert werden, dass Terminatoren absteigend nach ihrem Gewicht sortiert werden können (2 Punkte).

Hinweise zur Klasse *RobotFactory*

- Die Schlüssel-Werte-Paare des Assoziativspeichers **robots** beinhalten als Schlüssel die Seriennummer sowie als Wert den dazugehörigen Roboter
- Die Methode `Optional<T> getRobotBySerialNumber(serialNumber: String)` soll den Roboter zur eingehenden Seriennummer zurückgeben (4 Punkte)
- Die Methode `List<T> getRobotsByFabricationYear(fabricationYear: int)` soll alle Roboter zum eingehenden Fabrikationsjahr zurückgeben (4,5 Punkte)

Aufgabe 2 (16 Punkte)

Erstelle die JUnit-5-Testklasse **RobotFactoryTest** anhand des abgebildeten Klassendiagramms.

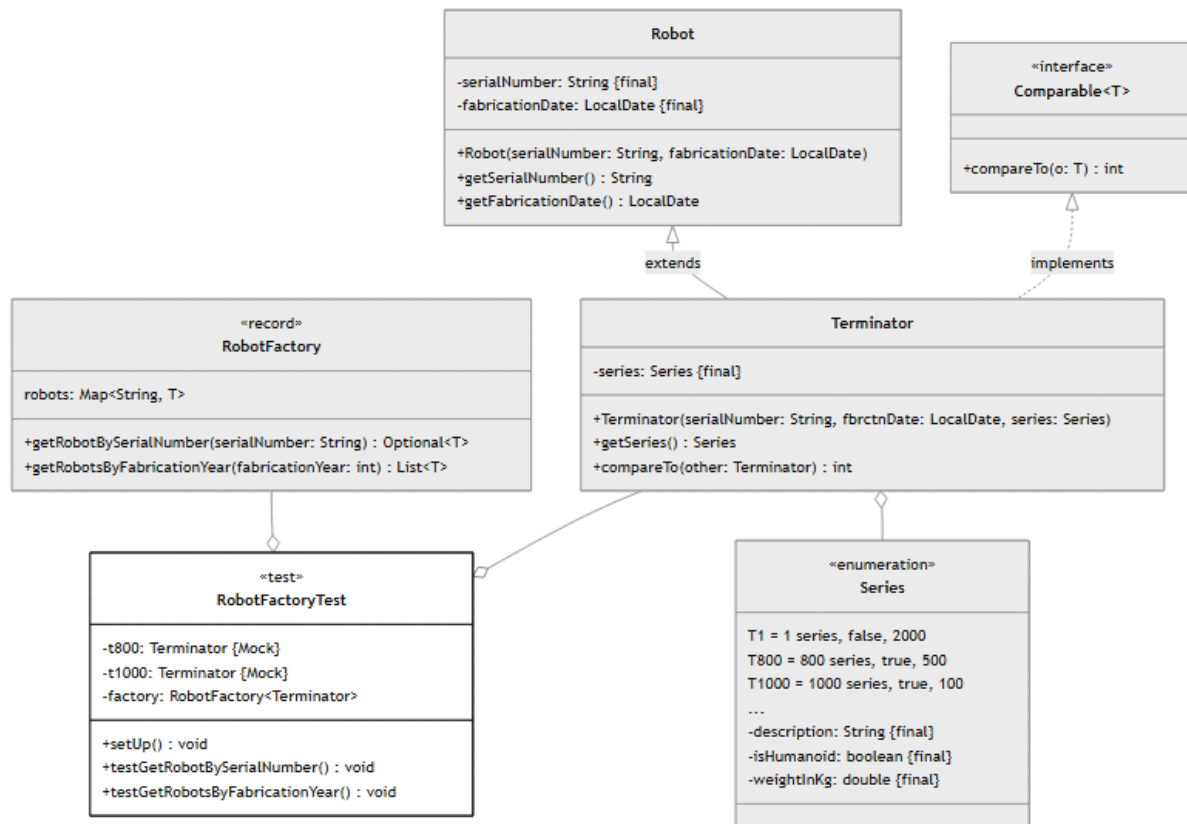


Abbildung 2: Klassendiagramm Aufgabe 2

Hinweise zur Klasse **RobotFactoryTest**

- Die Lebenszyklus-Methode **void setUp()** soll alle Mock-Objekte initialisieren, eine Roboterfabrik erstellen, diese Roboterfabrik dem entsprechenden Attribut zuweisen und in der Roboterfabrik für den Schlüssel **800-1** als Wert den Terminator T800 sowie für den Schlüssel **1000-1** als Wert den Terminator T1000 setzen (4,5 Punkte)
- Die Testmethode **void testGetRobotBySerialNumber()** soll prüfen, ob beim Aufruf der Methode **Optional<T> getRobotBySerialNumber(serialNumber: String)** auf die Roboterfabrik mit der Seriennummer **1000-0** ein leeres Optional und beim Aufruf mit der Seriennummer **1000-1** der Terminator T1000 als Optional zurückgegeben wird (4 Punkte)
- Die Testmethode **void testGetRobotsByFabricationYear()** soll prüfen, ob beim Aufruf der Methode **List<T> getRobotsByFabricationYear(fabricationYear: int)** auf die Roboterfabrik mit dem Wert **2029** eine Liste mit dem Terminator T800 als Eintrag zurückgegeben wird. Hierzu soll das Mock-Objekt T800 beim Aufruf der Methode **LocalDate getFabricationDate()** das Datum **24.03.2029** und das Mock-Objekt T1000 das Datum **12.07.2032** zurückgeben (4,5 Punkte)

Aufgabe 3 (19 Punkte)

Erstelle die Klasse **TerminatorQueries** anhand des abgebildeten Klassendiagramms.

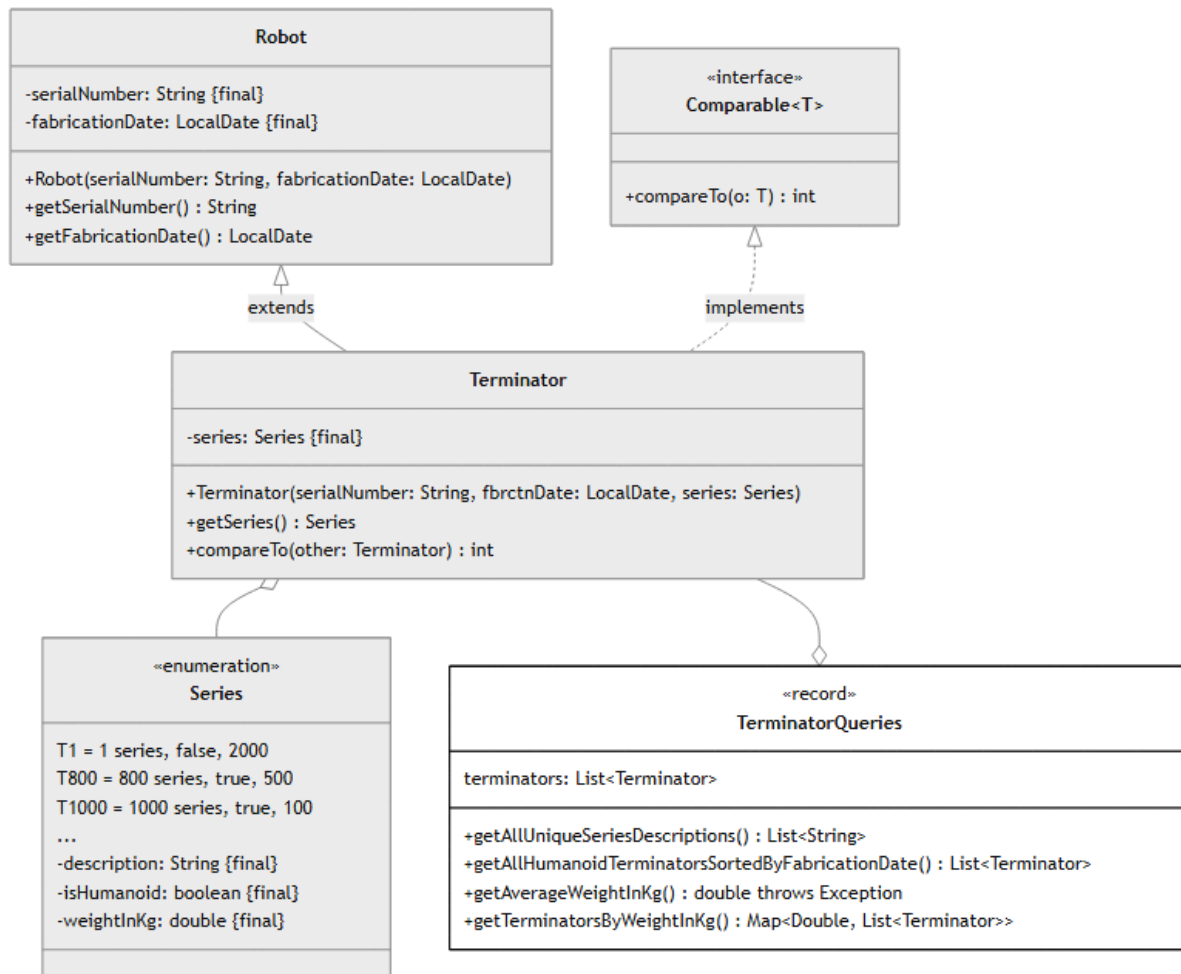


Abbildung 3: Klassendiagramm Aufgabe 3

Hinweise zur Klasse *TerminatorQueries*

- Die Methode **List<String> getAllUniqueSeriesDescriptions()** soll die Serienbeschreibungen aller Terminatoren ohne Dopplungen zurückgeben (3,5 Punkte)
- Die Methode **List<Terminator> getAllHumanoidTerminatorsSortedByFabricationDate()** soll alle menschenähnlichen Terminatoren absteigend sortiert nach ihrem Fabrikationsdatum zurückgeben (5 Punkte)
- Die Methode **double getAverageWeightInKg() throws Exception** soll das durchschnittliche Gewicht aller Terminatoren zurückgeben. Für den Fall, dass kein durchschnittliches Gewicht berechnet werden kann (leere Liste), soll die Ausnahme **Exception** ausgelöst werden (6 Punkte)
- Die Methode **Map<Double, List<Terminator>> getTerminatorsByWeightInKg()** soll alle Terminatoren gruppiert nach ihrem Gewicht zurückgeben (3,5 Punkte)

Cheatsheet

java.lang.Boolean

compare(x: boolean, y: boolean)	int	S
valueOf(s: String)	Boolean	S
valueOf(b: boolean)	Boolean	S

java.lang.Comparable<T>

compareTo(o: T)	int	
-----------------	-----	--

java.lang.Double

compare(d1: double, d2: double)	int	S
valueOf(s: String)	Double	S
valueOf(d: double)	Double	S

java.lang.Integer

compare(x: int, y: int)	int	S
valueOf(s: String)	Integer	S
valueOf(i: int)	Integer	S

java.lang.Long

compare(x: long, y: long)	long	S
valueOf(s: String)	Long	S
valueOf(l: long)	Long	S

java.lang.Object

equals(object: Object)	boolean	
hashCode()	int	
toString()	String	

java.lang.Runnable

run()	void	
-------	------	--

java.lang.String

charAt(index: int)	char	
length()	int	
split(regex: String)	String[]	
toLowerCase()	String	
toUpperCase()	String	

java.lang.System

currentTimeMillis()	long	S
---------------------	------	---

java.io.PrintStream

print(obj: Object)	void	
printf(String format, Object... args)	PrintStream	
println()	void	
println(x: Object)	void	

java.util.Collections

sort(list: List<T>)	void	S
sort(list: List<T>, c: Comparator<T>)	void	S

java.util.Comparator<T>

compare(o1: T, o2: T)	int	
comparing(keyExtractor: Function<T,U>)	Comparator<T>	S

java.util.Entry<K,V>

getKey()	K	
getValue()	V	
setValue(value: V)	V	

java.util.Enumeration

valueOf(arg0: String)	Enumeration	S
values()	Enumeration[]	S

java.util.List<E>

add(e: E)	boolean	
add(index: int, element: E)	void	
contains(o: Object)	boolean	
forEach(action: Consumer<T>)	void	
get(index: int)	E	
of(elements: E...)	List<E>	S
remove(index: int)	E	
remove(o: Object)	boolean	
reversed()	List<T>	
size()	int	
sort(c: Comparator<T>)	void	

java.util.Map<K,V>

containsKey(key: Object)	boolean	
containsValue(value: Object)	boolean	
entrySet()	Set<Entry<K,V>>	
forEach(action: BiConsumer<K,V>)	void	
get(key: Object)	V	
keySet()	Set<K>	
put(key: K, value: V)	V	
putIfAbsent(key: K, value: V)	V	
values()	Collection<V>	

java.util.Optional<T>

empty()	Optional<T>	S
get()	T	
ifPresent(action: Consumer<T>)	void	
ifPresentOrElse(action: Consumer<T>, emptyAction: Runnable)	void	
isPresent()	boolean	
of(t: T)	Optional<T>	S
ofNullable(t: T)	Optional<T>	S
orElse(other: T)	T	

java.util.OptionalDouble

empty()	OptionalDouble	S
getAsDouble()	double	
ifPresent(action: DoubleConsumer)	void	
ifPresentOrElse(action: DoubleConsumer, emptyAction: Runnable)	void	
isPresent()	boolean	
of(value: double)	OptionalDouble	S
orElse(other: double)	double	

java.util.Random

nextInt(origin: int, bound: int)	int	
----------------------------------	-----	--

java.util.function.BiConsumer

accept(t: T, u: U)	void	
--------------------	------	--

java.util.function.Consumer<T>

accept(t: T)	void	
--------------	------	--

java.util.function.DoubleConsumer

accept(value: double)	void	
-----------------------	------	--

java.util.function.Function<T,R>

apply(t: T)	R	
-------------	---	--

java.util.function.Predicate<T>

test(t: T)	boolean	
------------	---------	--

java.util.function.ToDoubleFunction<T,R>

applyAsDouble(value: T)	double	
-------------------------	--------	--

java.util.function.ToIntFunction<T,R>

applyAsInt(value: T)	int	
----------------------	-----	--

java.time.LocalDate

getDayOfMonth()	int	
getDayOfYear()	int	
getMonth()	Month	
getMonthValue()	int	
getYear()	int	
now()	LocalDate	S
of(year: int, month: int, dayOfMonth: int)	LocalDate	S
parse(text: CharSequence)	LocalDate	S

java.time.LocalTime

getHour()	int	
getMinute()	int	
getSecond()	int	
now()	LocalTime	S
of(hour: int, minute: int, second: int)	LocalTime	S
parse(text: CharSequence)	LocalTime	S

java.util.stream.Collectors

averagingDouble(mapper: ToDoubleFunction<T>)	Collector<T,?,Double>	S
counting()	Collector<T,?,Long>	S
groupingBy(classifier: Function<T,K>)	Collector<T,?,Map<K,List<T>>>	S
joining(delimiter: CharSequence)	Collector<CharSequence,?,String>	S
mapping(m: Function<T,U>, ds: Collector<U,A,R>)	Collector<T,?,R>	S
partitioningBy(predicate: Predicate<T>)	Collector<T,?,Map<Boolean,List<T>>>	S
summingDouble(mapper: ToDoubleFunction<? super T>)	Collector<T,?,Double>	S
toList()	Collector<T,?,List<T>>	S
toMap(keyM: Function<T,K>, valueM: Function<T,U>)	Collector<T,?,Map<K,U>>	S
toSet()	Collector<T,?,Set<T>>	S

java.util.stream.DoubleStream

average()	OptionalDouble	
sum()	double	

java.util.stream.IntStream

average()	OptionalDouble	
sum()	int	

java.util.stream.Stream<T>

allMatch(predicate: Predicate<T>)	boolean	
anyMatch(predicate: Predicate<T>)	boolean	
collect(collector: Collector<T,A,R>)	R	
count()	long	
distinct()	Stream<T>	
filter(predicate: Predicate<T>)	Stream<T>	
findAny()	Optional<T>	
findFirst()	Optional<T>	
flatMap(mapper: Function<T,Stream<R>>)	Stream<R>	
forEach(action: Consumer<T>)	void	
limit(maxSize: long)	Stream<T>	
map(mapper: Function<T,R>)	Stream<R>	
mapToDouble(mapper: ToDoubleFunction<T,R>)	DoubleStream	
mapToInt(mapper: ToIntFunction<T,R>)	IntStream	
max(comparator: Comparator<T>)	Optional<T>	
min(comparator: Comparator<T>)	Optional<T>	
noneMatch(predicate: Predicate<T>)	boolean	
skip(n: long)	Stream<T>	
sorted()	Stream<T>	
sorted(comparator: Comparator<T>)	Stream<T>	
toList()	List<T>	

org.junit.jupiter.api.Assertions

assertEquals(expected: Object, actual: Object)	void	S
assertFalse(condition: boolean)	void	S
assertNotEquals(expected: Object, actual: Object)	void	S
assertNotNull(actual: Object)	void	S
assertNotSame(expected: Object, actual: Object)	void	S
assertNull(actual: Object)	void	S
assertSame(expected: Object, actual: Object)	void	S
assertThrows(expectedType: Class<T>, executable: Executable)	T	S
assertTrue(condition: boolean)	void	S

org.junit.jupiter.api.Executable

execute()	void	
-----------	------	--

org.mockito.Mockito

when(methodCall: T)	OngoingStubbing<T>	S
---------------------	--------------------	---

org.mockito.MockitoAnnotations

openMocks(testClass: Object)	AutoCloseable	S
------------------------------	---------------	---

org.mockito.stubbing.OngoingStubbing<T>

thenReturn(value: T)	OngoingStubbing<T>	
----------------------	--------------------	--